Encapsulation

↓

to form a Capsule

Animal

- No of legs, type --- (data)
- eat(), walk, run(), --- (behaviour)

```
Class Animal {
    int noOfLegs;
    String type;
    void eat() {
        =
    }
    void run() {
        =
    }
}
```

int noOfLegs; String type; → data

eat(), run() → Behaviour

→ Encapsulation

- ① Both data & Behaviour should be together (encapsulated) within an Object.
- ② we should not provide unrestricted access of data.

BankAccount

- balance (data)
- bankName, ifsc, ---

→ deposit
→ withdraw

(Behaviour)

```
Class BankAccount {
    double balance;
    String bankName;
    String ifsc;
    |
    |
}
```

```
BankAccount ba = new BankAccount();
ba.balance = 1,00,000; (X)
```

Access Modifiers

- who have access to
  - → variable
  - → Method
  - → constructor
  - → class

Access Modifiers

- public
- default
- protected
- private

```
Class BankAccount {
    private double balance;
}
```

Private

```
Class BA {
    private double balance;

    void withdraw() {
        balance--;
    }
}
```

default

↳ Anyone can access belonging to same package;

```
public Class Demo {
    void main() {
    }
}

Class Animal {
    =
}
```

→ A package group similar classes / Interfaces together.

Protected

↳ package + Inherited class (child class)

public

↳ Anyone can access.

private | default | protected | public

```
public class Demo {
    void main() {
    }
}

class Student {
}
```

```
class BankAccount {
    private double balance;
    public void deposit(int amount){
        balance += amount;
    }
    public void withdraw(int amount){
        balance -= amount;
    }
    public double getBalance() {
        return balance;
    }
}
```

```
BankAccount ba = new BankAccount();
ba.balance X
ba.deposit(500);
ba.withdraw(300);
ba.getBalance();
```

getters / setters

Packages

User-defined package

In-built package

```
import java.util.ArrayList;
public class Demo {
    void main() {
        ==
    }
}
```

Demo.java

→ Bytecode → Demo.class

java.util
java.lang → String
java.io
!

Inheritance (↘)

Vehicle → start
↓ (is-a)
Car → start

Student
↓ (is-a)
EngineeringStudent

```
Class Vehicle {
    void start () {
        ///
    }
}
```

↓ (is-a)

```
Class Car extends Vehicle {
    void setGear() {..}
}
```

```
Car c = new Car();
c.start();
```

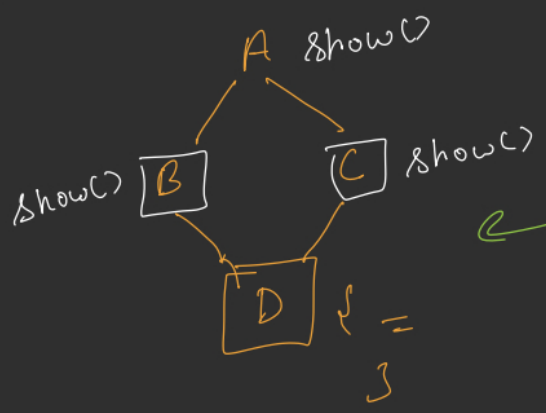
Inheritance

- Code Reusability
- Support polymorphism

Why Multiple Inheritance Not supported in Java?

⇓

Due to Diamond problem




```
D d = new D();
d.show();
```

interfaces

Super → Reference of Parent Object

↓

this → Reference of Current Object

Uses of super

① To access parent class variable

```
class A {
    int x = 4;
}

class B extends A {
    int x = 5;
    super.x
}
```

②

```
class A {
    void great () {
    }
}

class B {
    void sayHello() {
        super.great();
    }
}
```

→ Can call parent class method using super.

③ Call parent class Constructor

```
class A {
        A() {
        }
}

class B extends A {
        B() {
            super();
        }
}
```